

Programare Java

Java Vs. Java Script

Java

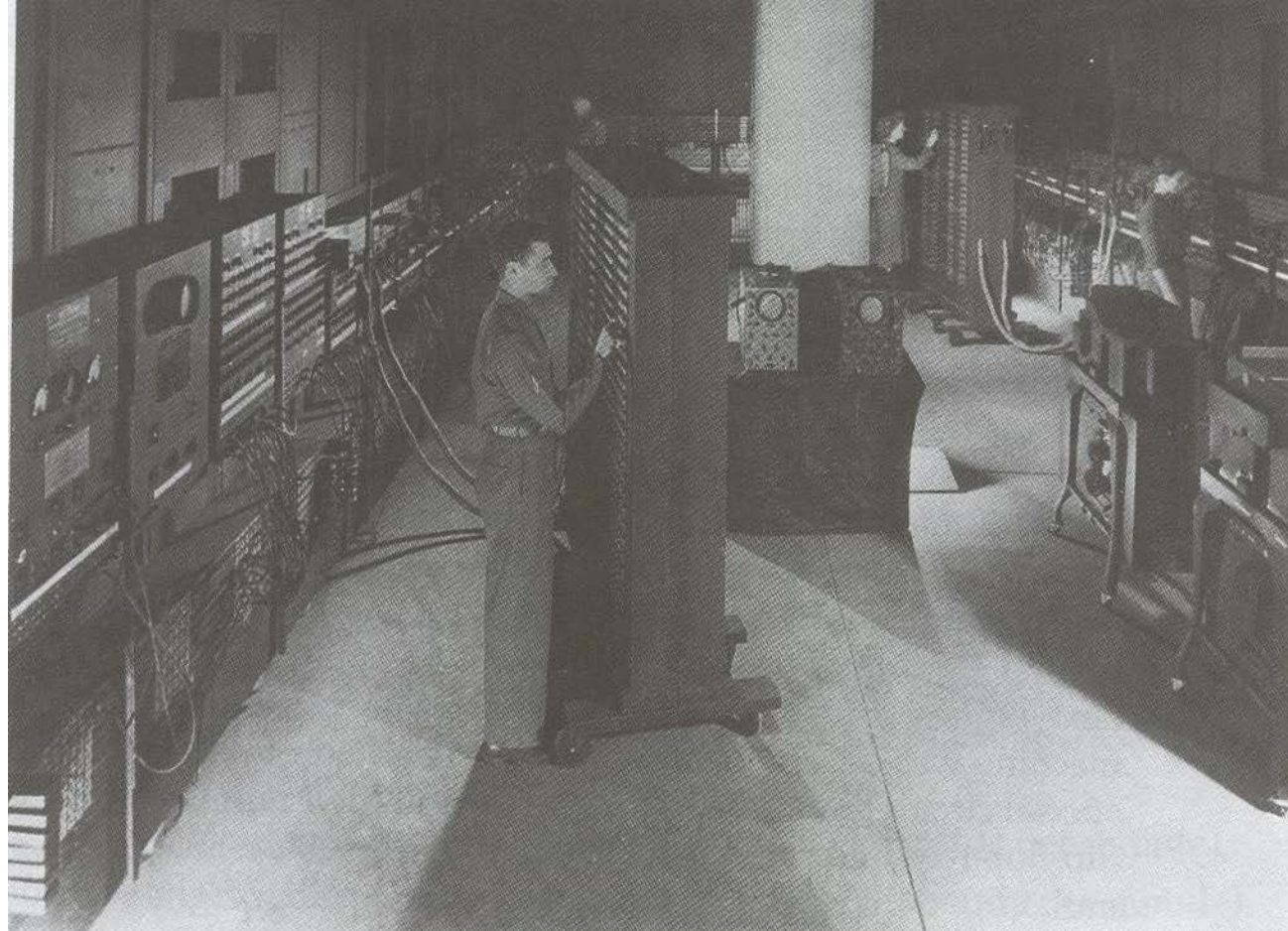
- Un limbaj de programare complet dezvoltat de Sun
- Poate fi folosit pentru a dezvolta software bazat pe web sau independent
- Sunt disponibile multe biblioteci de coduri pre-create si gata compilate
- Pentru programe mai complexe și mai puternice

Java Script

- O limbaj mai mic, utilizat în principal pentru aplicații bazate pe web (rulează printr-un browser web precum Internet Explorer, Firefox, Safari, Chrome)
- Bun pentru programarea efectelor speciale simple pentru pagina. web, de exemplu, roll-over-uri

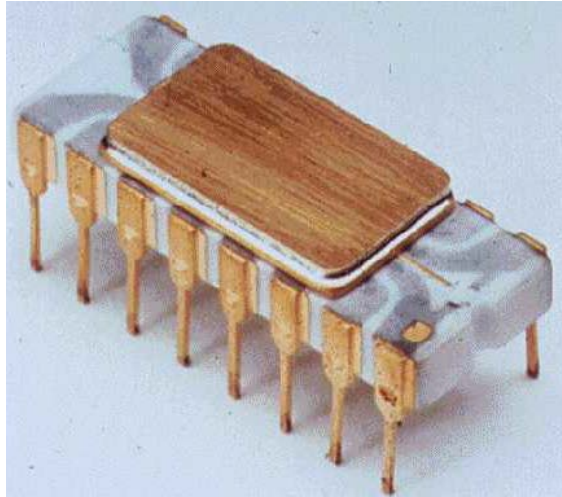
Java: Istoric

Computerele în trecut



Java: Istoric (2)

Invenția microprocesorului a revoluționat computerele



Intel microprocessor



Commodore Pet microcomputer

Java: Istoric (3)

S-a crezut că următorul pas logic pentru microprocesoare a fost să le facă să ruleze pe electronice inteligente de larg consum



Java Istoric (4)

Sun Microsystems a finanțat un proiect intern de cercetare „Green” pentru a investiga această oportunitate.

Rezultat: Un limbaj de programare numit „Oak”



Blatant advertisement: James Gosling was a graduate of the U of C Computer Science program.

Java Istoric (5)

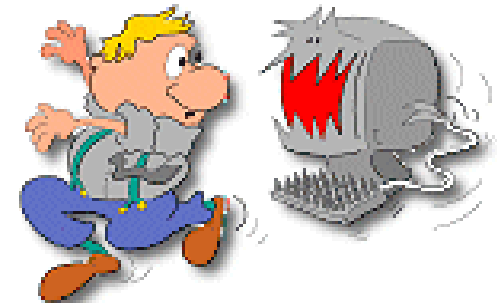
- Problemă: exista deja un limbaj de programare numit Oak.
 - Echipa „Green” s-a întâlnit la o cafenea locală pentru a găsi un alt nume ...Java!
-



Java: Istoric (6)

Conceptul de dispozitive inteligente nu a prins.

Proiectul Green și lucrul la limbajul Java au fost aproape anulate.



Java: Istoric (7)

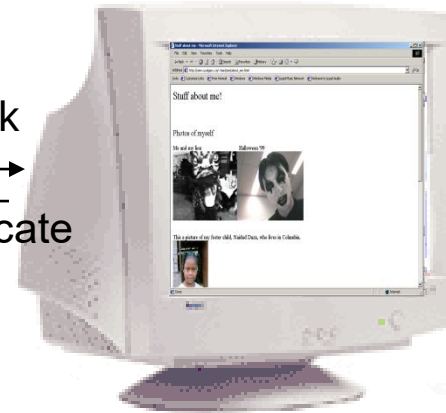
Popularitatea internetului a dus la re-concentrarea utilizării Java de către Sun pe computere.

Înainte de apariția Java, paginile web permiteau să descărcați doar text și imagini.

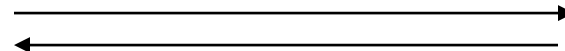
**Computerul de acasă
rulează un browser web**



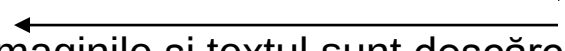
**Server care conține o
pagină web**



Utilizatorul face click pe un link



Imaginile și textul sunt descărcate



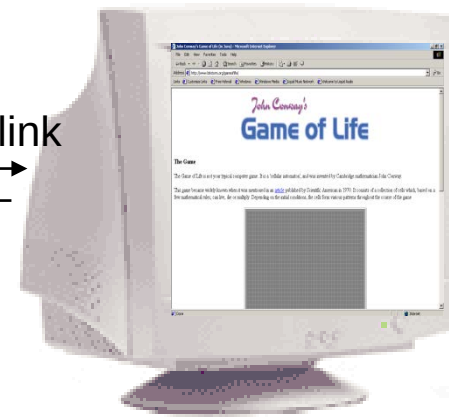
Java: Istoric (8)

Browsers web ce au activat Java au permis descărcarea de programe (Applets).

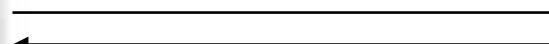
**Computerul de acasă rulează
un browser web**



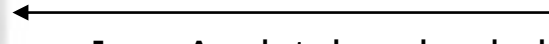
**Server care conține o
pagină web**



Utilizatorul face click pe un link

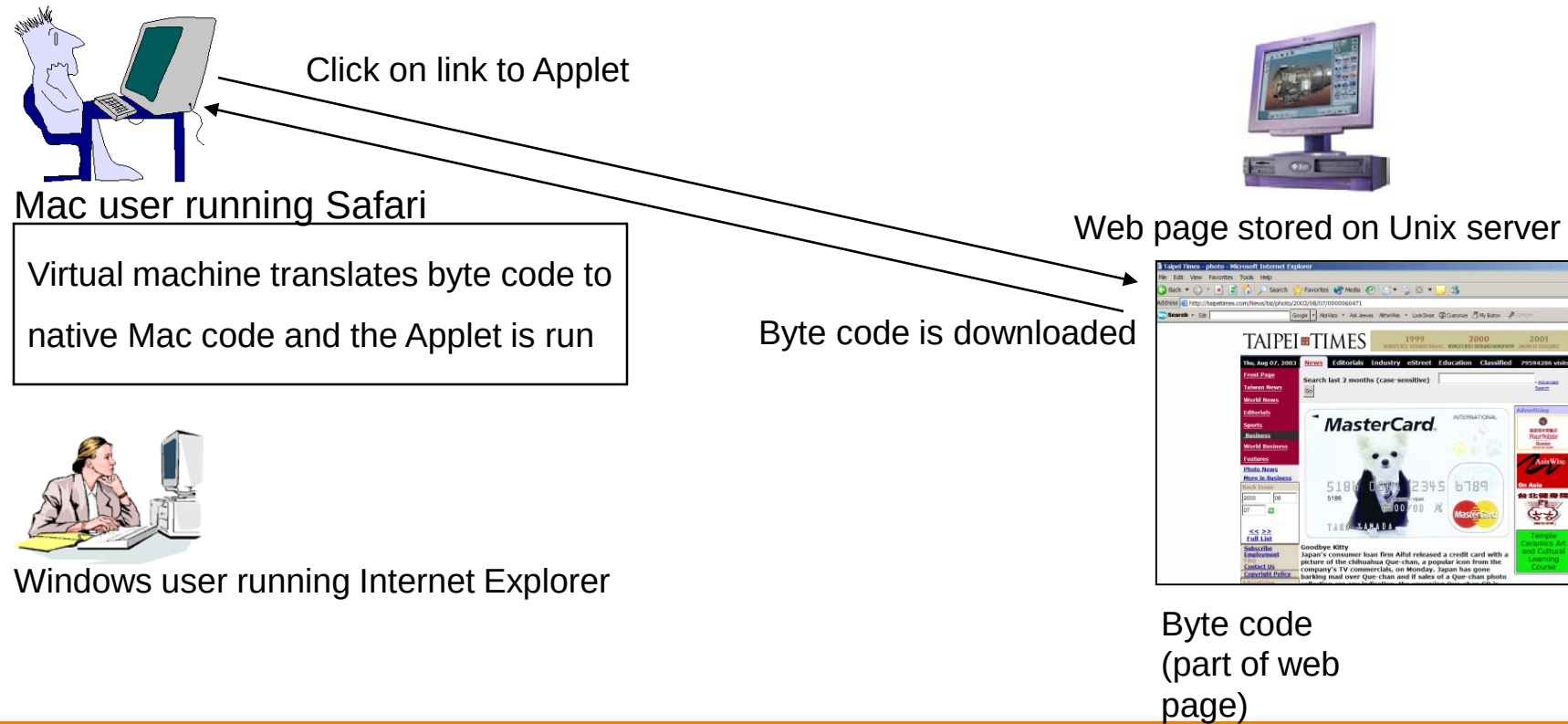


Java Applet downloaded



Java: Write Once, Run Anywhere

Consecința istoricului Java:
independența platformei



Java: Write Once, Run Anywhere

Consequence of Java's Istoric:
platform-independent



Mac user running Safari



Web page stored on Unix server

Click on link to Applet

Byte code is downloaded



Windows user running Internet Explorer



Virtual machine translates byte code to
native Windows code and the Applet is run

Java: Write Once, Run Anywhere (2)

But Java c

o based) p



Dungeon Master (Java version)

<http://homepage.mac.com/aberfield/dmj/>



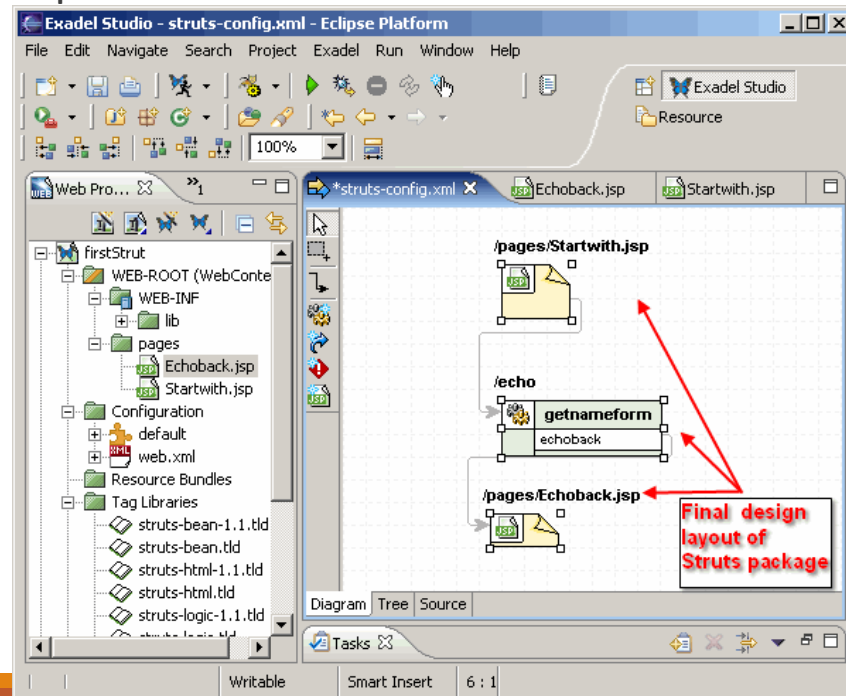
Kung Fu Panda 2: THQ

Java: Write Once, Run Anywhere (3)

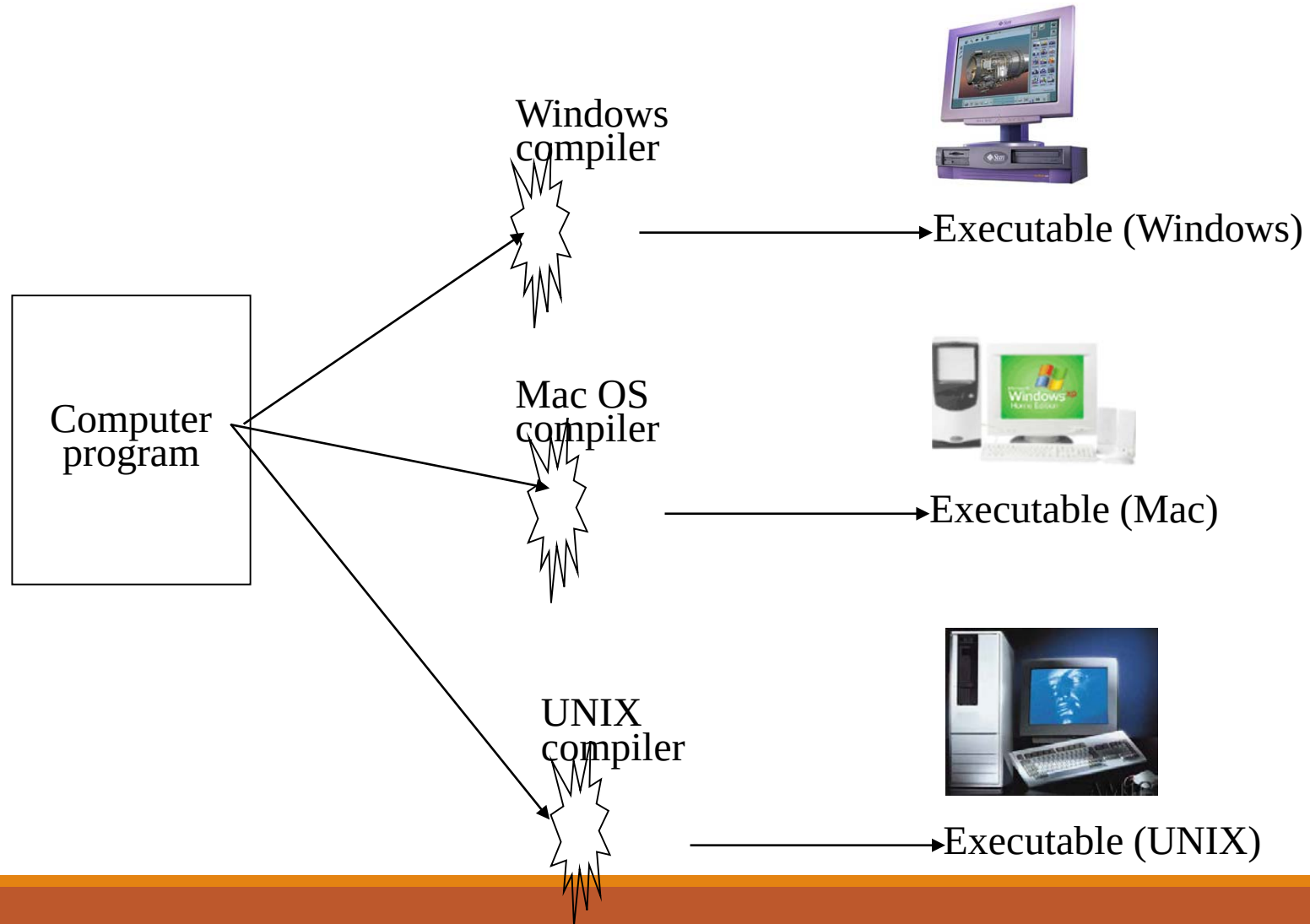
Java has been used by large and reputable companies to create serious standalone applications.

Example:

- Eclipse¹: started as a programming environment created by IBM for developing Java programs. The program Eclipse was itself written in Java.



Operating Systems



Scurt istoric

scopul declarat de a dezvolta un software performant pentru aparatele electronice de larg consum: mici, portabile, distribuite și lucrând în timp real.

Limbajul luat inițial în considerare a fost C++.

firma Sun a pornit proiectarea unui nou limbaj de programare asemănător cu C++ dar mult mai flexibil, mai simplu și mai portabil

Istoricul limbajului Java

1991 – firma Sun începe dezvoltarea unui proiect software pentru aparatură electronică inteligentă care să poată fi conectată în rețea

1992 – începe dezvoltarea unui nou limbaj de programare care inițial s-a numit OAK

1993 – primul browser www care s-a numit Mosaic; de asemenea a fost disponibil și un web browser, numit webrunner capabil să ruleze programe scrise în OAK;

1994 – OAK a devenit Java și webrunner a devenit HotJava

1995 – Netscape integrează Java în Netscape Navigator 2.0

1996 – apare Java 1.0

1997 – Java 1.1, HotJava 1.0 și JavaOS 1.0

1998 – Java Foundation Classes , JVC ce include Swing 1.0

După 1999 – apar noi versiuni de Java

Acum sunt versiuni JDK 1.8.x

Caracteristici ale limbajului

limbaj **simplu** de folosit chiar și de către programatorii neprofesioniști.

Limbajul Java este independent de arhitectura calculatorului pe care lucrează și foarte **portabil**. Sloganul limbajului este "Write once run anywhere", adică pe orice calc unde e instalat Java

Execuția aplicațiilor Java este **interpretată**.

Limbajul Java este în totalitate **orientat obiect**. Elimină complet stilul de programare procedural, se bazează pe **moștenire, încapsulare, polimorfism**

Java este **distribuit**, având implementate biblioteci pentru lucrul în rețea care ne oferă TCP/IP, URL și încărcarea resurselor din rețea.

Java este **robust**. Elimină sursele de erori, odată ce a trecut de compilare, nu se mai întâmplă ca un program să se blocheze

Java este un limbaj cu **securitate ridicată**, fiind unul dintre cele mai sigure limbaje de programare

lucrează cu **mai multe fire de execuție**

Java este **dinamic**.

Reprezentarea informațiilor cu obiecte

Obiecte

Informațiile pe care le reprezentăm în memoria calculatorului pot fi uneori complexe

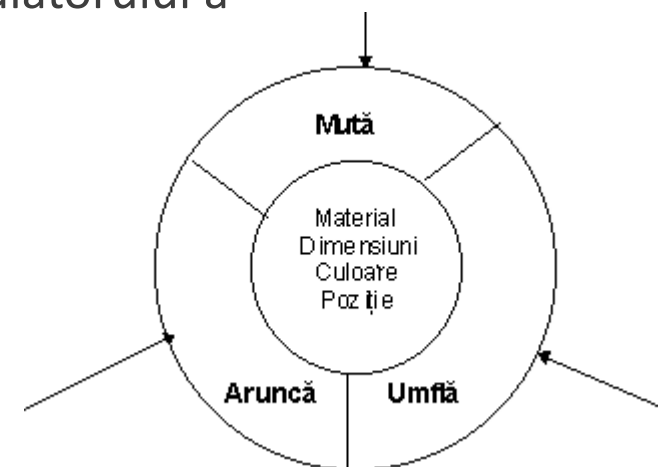
Un obiect este caracterizat prin **date** și **comportament**

Ex de obiect: o minge

Din punctul de vedere al programării, un **obiect** este o reprezentare în memoria calculatorului a **proprietăților** și **comportamentului**.

Încapsularea informațiilor în interiorul obiectelor

procesul de încapsulare este procesul de ascundere a detaliilor neimportante sau sensibile de construcție a obiectului.

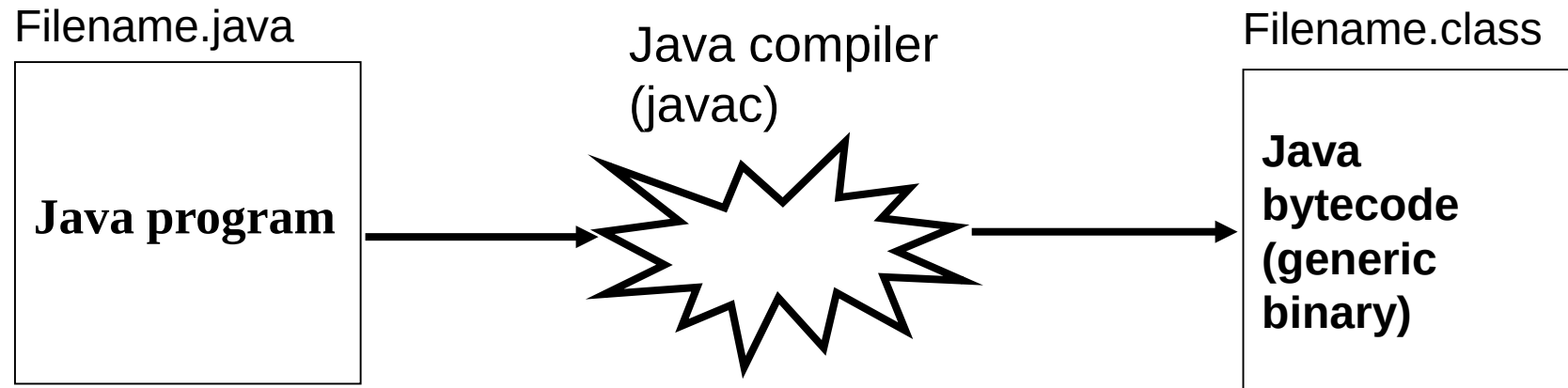


Clase de obiecte

Încapsularea informațiilor în interiorul obiectelor

O vedere la nivel înalt a traducerii / executării programelor Java

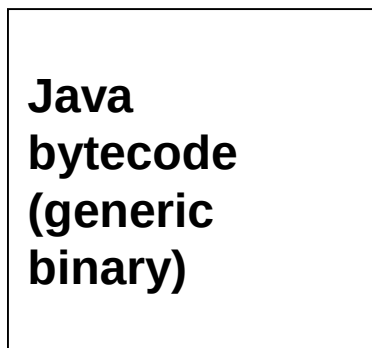
Etapa1: Compilation



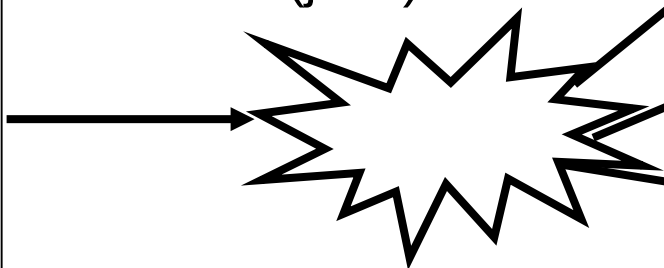
O vedere la nivel înalt a traducerii / executării programelor Java (2)

Etape 2: Interpretarea și executarea byte code

Filename.class



Java interpreter
(java)



Machine language
instruction (UNIX)

Machine language
instruction (Windows)

Machine language
instruction (Apple)

Ce versiune Java?

Java 6+ JDK (Java Development Kit), Standard Edition include:

- JDK (Java development kit) – pentru *dezvoltare programe* Java.
- JRE (Java Runtime environment) – doar pentru execuție *running* programe Java.
 - Java Plug-in – o versiune specială a JRE concepută pentru a rula prin intermediul browserelor web.

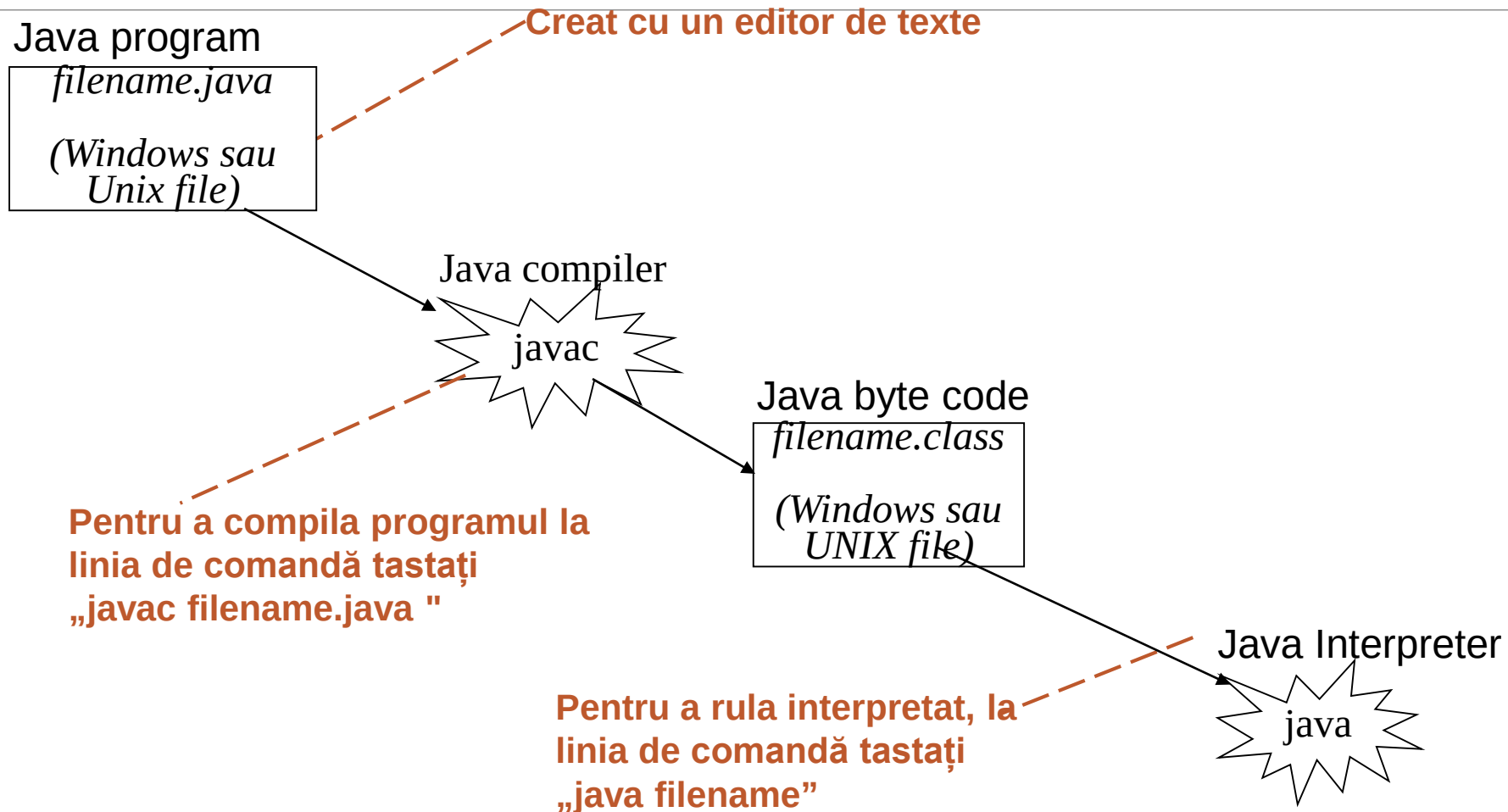
<http://java.sun.com/javase/downloads/index.jsp>

Cel mai mic program Java compilabil și executabil

Numele exemplului este : Smallest.java (*Notă importantă: numele fișierului se potrivește cu cuvântul după cuvântul cheie „class”*)

```
public class Smallest
{
    public static void main (String[] args)
    {
    }
}
```

Crearea, compilarea și rularea programelor Java



Compilarea

Smallest.java

```
public class Smallest
{
    public static void main (String[] args)
    {
    }
}
```

javac

tastati "javac
Smallest.java"

Smallest.class

(Java byte code)

```
100001000000001000
001001000000001001
```

```
:      :
```

Rularea programului

Smallest.class

(Java byte code)

```
100001000000001000  
001001000000001001
```

: :

java

Tastati "java
Smallest"

(Cod specific unei anumite platforme /SC

```
101001110000001000  
00100111001111001
```

: :



Running The Java Compiler At Home

După instalarea Java, va trebui să indicați sistemului de operare unde a fost instalat compilatorul Java („seth path”).

Pentru detalii despre cum să setați variabila de cale pentru sistemul de operare, încercați site-ul web Sun sau Java.

Exemplu despre cum să setați calea în Windows:

<http://java.sun.com/j2se/1.4.2/install-windows.html>

Observații

Fiecare instrucțiune Java trebuie să fie urmată de un punct și virgulă!

Format general

Instruction1;

Instruction2;

Instruction3;

: :

Example

int num = 0;

System.out.println(num);

: :

Exemplu: Hello World Program

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

Totul este într-o clasă

Un fișier, o clasă publică

În clasa publică executabilă:

- **public static void main(String [] args)**

Tipuri primitive de date

boolean

char

byte

short

int

long

float

double

Reali speciali definiți de IEEE

NaN

POSITIVE_INFINITY, NEGATIVE_INFINITY, NaN.

MIN_VALUE și MAX_VALUE.

Tipuri referință

Sunt folosite pentru a referi un obiect din interiorul unui alt obiect.

Valoare implicită este definită de către limbajul Java prin cuvântul rezervat **null**.

Rezervarea obiectului trebuie făcută explicit în program printr-o expresie de alocare care folosește cuvântul rezervat **new**.

Tipul referință către o clasă

trimite către o instanță a unei clase de obiecte.

Exemple

Minge m1;

Student s1;

String sir;

Tipul referință către o interfață

Tipul *referință către o interfață* permite păstrarea unor referințe către obiecte care respectă o anumită interfață.

Tipul referință către un tablou

Tipul referință către un tablou este un tip referință care poate păstra o trimitere către locația din memorie a unui tablou de elemente.

Tablourile sunt de asemenea o clasă

```
long [] a = new long[5];
```

- Puteți obține lungimea cu ajutorul câmpului de lungime al obiectului matrice a, ca acesta :
a.length

Clasa String este foarte frecvent utilizată pentru a reprezenta șiruri de caractere, de exemplu

```
String s1 = "Hello ", s2 = "Wolrd!";  
String s3 = s1 + s2;
```

Operatori (la fel ca în C/C++)

++,-- Auto increment/decrement

+, - plus/minus ca operatori unari

*, / Înmulțire/împărțire

% rest

+, - Adunare/Scădere

Declararea variabilelor

Sintaxa:

Tip NumeVariabilă [, NumeVariabilă];

Exemple:

```
int n = 1;
```

```
char ch = 'A';
```

```
String s = "Hello";
```

```
Long L = new Long(100000);
```

```
boolean done = false;
```

```
final double pi = 3.14159265358979323846;
```

```
Employee joe = new Employee();
```

```
char [] a = new char[3];
```

```
Vector v = new Vector();
```

Clasa de memorare

Variabile locale

Variabilele locale sunt declarate, rezervate în memorie și utilizate doar în interiorul unor blocuri de instrucțiuni, fiind distruse automat la ieșirea din aceste blocuri.

Variabile statice

Se declară cu **static**

se mai numesc uneori și *variabile de clasă*.

Variabile dinamice

Aceste variabile pot fi alocate la cerere, dinamic, în orice moment al execuției programului. Ele vor fi distruse doar atunci când nu mai sunt referite de nimeni.

Se utilizează o expresie de alocare care folosește cuvântul rezervat **new**

```
Minge mingeaMea = new Minge( );
```

```
Minge echipament[] = new Minge[5];
```

```
int numere[] = new int[10];
```

Tablouri de variabile

Declarația variabilelor de tip tablou

Tablouri cu 1 dimesiune

```
int[ ] numere;
```

```
int numere[ ];
```

Tablouri cu 2 dimensiuni (matrice)

```
float [ ][ ] matrice;
```

```
float[ ] matrice[];
```

```
float matrice[][];
```

Alocarea spațiului pentru elementele tabloului trebuie făcută explicit.

Pentru tablourile cu mai multe dimensiuni, rezervarea spațiului se poate face cu următoarea sintaxă:

```
byte [][ ] octeti = new byte[23][5];
```

Inițializarea tablourilor.

```
char []caractere = { 'a', 'b ', 'c ', 'd ' };
```

```
int [][]numere = {
```

```
{ 1, 3, 4, 5 },
```

```
{ 2, 4, 5 },
```

```
{ 1, 2, 3, 4, 5 }
```

```
};
```

```
double [][][]reali = {
```

```
{ { 0.0, -1.0 }, { 4.5 } },
```

```
{ { 2.5, 3.0 } }
```

```
};
```

Lungimea tablourilor

```
float []tablou = new float[25];
```

```
int dimensiune = tablou.length;    // dimensiune primește valoarea 25
```

sau

```
float [][]multiTablou = new float[3][4];
```

```
int dimensiune1 = multiTablou[2].length;    // dimensiune1 primește valoarea 4
```

```
int dimensiune2 = multiTablou.length;       // dimensiune2 primește valoarea 3
```

Referirea elementelor din tablou

```
int []tablou = new int[10];  
tablou[3] = 1;  
// al patrulea element primește valoarea 1  
float [][]reali = new float[3][4];  
reali[2][3] = 1.0f;  
// al patrulea element din al treilea tablou  
// primește valoarea 1  
char [][]caractere = new char [5][];
```

Caractere



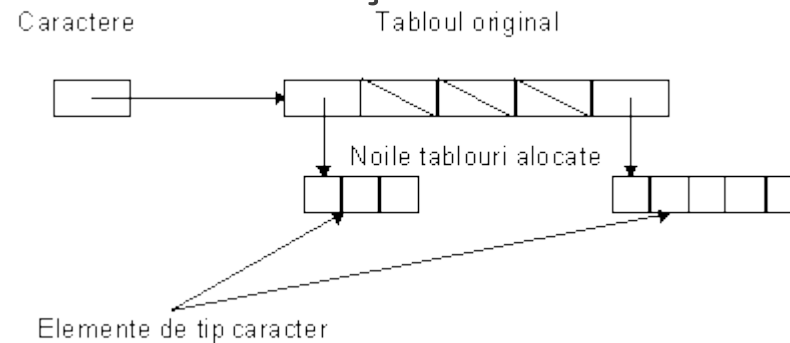
Tabloul de referințe



Variabila referință numită caractere conține deocamdată un tablou de 5 referințe la tablouri de caractere. Cele cinci referințe sunt inițializate cu null. Putem inițializa aceste tablouri prin atribuiri de expresii de alocare:

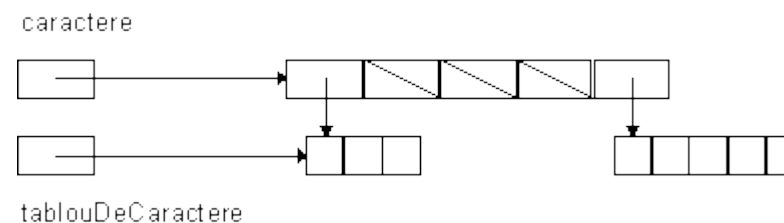
```
caractere[0] = new char [3];
```

```
caractere[4] = new char [5];
```



La fel, putem scrie:

```
char []tablouDeCaractere = caractere[0];
```



Alocarea și eliberarea tablourilor

exemplu:

```
float []reali = new float[10];
```

```
reali = null; // eliberarea tabloului
```

sau

```
reali = new float[15]; // eliberarea în alt fel
```

sau

```
{
```

```
float []reali = new float[10];
```

Conversii

Conversii de extindere a valorii

În aceste conversii valoarea se reprezintă într-o zonă mai mare fără să se piardă nici un fel de informații. Iată conversiile de extindere pe tipuri primitive:

- byte la short, int, long, float sau double
- short la int, long, float sau double
- char la int, long, float sau double
- int la long, float sau double
- long la float sau double
- float la double

Conversii de trunchiere a valorii

Convențiile de trunchiere a valorii pot produce pierderi de informație pentru că ele convertesc tipuri mai bogate în informații către tipuri mai sărace. Conversiile de trunchiere pe tipurile elementare sunt următoarele:

- byte la char
- short la byte sau char
- char la byte sau short
- int la byte, short sau char
- long la byte, short char, sau int
- float la byte, short, char, int sau long
- double la byte, short, char, int, long sau float.

Conversii pe tipuri referință

Limbajul Java definește extrem de strict conversiile posibile în cazul tipurilor referință pentru a salva programatorul de eventualele necazuri care pot apare în timpul execuției. Iată conversiile posibile:

- O referință către un obiect aparținând unei clase C poate fi convertit la o referință către un obiect aparținând clasei S doar în cazul în care C este chiar S sau C este derivată direct sau indirect din S.
- O referință către un obiect aparținând unei clase C poate fi convertit către o referință de interfață I numai dacă clasa C implementează interfața I.
- O referință către un tablou poate fi convertită la o referință către o clasă numai dacă clasa respectivă este clasa Object.
- O referință către un tablou de elemente ale cărui elemente sunt de tipul T1 poate fi convertită la o referință către un tablou de elemente de tip T2 numai dacă T1 și T2 reprezintă același tip primitiv sau T2 este un tip referință și T1 poate fi convertit către T2.

Conversii la operația de atribuire

	boolean	char	byte	short	int	long	float	double
boolean	Da	Nu	Nu	Nu	Nu	Nu	Nu	Nu
char	Nu	Da	Da	Da	Nu	Nu	Nu	Nu
byte	Nu	Da	Da	Nu	Nu	Nu	Nu	Nu
short	Nu	Da	Da	Da	Nu	Nu	Nu	Nu
int	Nu	Da	Da	Da	Da	Nu	Nu	Nu
long	Nu	Da	Da	Da	Da	Da	Nu	Nu
float	Nu	Da	Da	Da	Da	Da	Da	Nu
double	Nu	Da	Da	Da	Da	Da	Da	Da

Conversiile posibile la atribuirea unei valori de tipul T la o variabilă de tipul S.

	T este o clasă care nu este finală	T este o clasă care este finală	T este o interfață	T = B[] este un tablou cu elemente de tipul B
S este o clasă care nu este finală	T trebuie să fie subclasă a lui S	T trebuie să fie o subclasă a lui S	eroare la compilare	S trebuie să fie Object
S este o clasă care este finală	T trebuie să fie aceeași clasă ca și S	T trebuie să fie aceeași clasă ca și S	eroare la compilare	eroare la compilare
S este o interfață	T trebuie să implementeze interfața S	T trebuie să implementeze interfața S	T trebuie să fie o subinterfață a lui S	eroare la compilare
S = A[] este un tablou cu elemente de tipul A	eroare la compilare	eroare la compilare	eroare la compilare	A sau B sunt același tip primitiv sau A este un tip referință și B poate fi atribuit lui A

Conversii explicite

Conversiile de tip cast, sau casturile, sunt apelate de către programator în mod explicit. Sintaxa pentru construcția unui cast este scrierea tipului către care dorim să convertim în paranteze în fața valorii pe care dorim să o convertim. Forma generală este:

(Tip) Valoare

Cazurile posibile la convertirea unei referințe de tip T într-o referință de tip S

	T este o clasă care nu este finală	T este o clasă care este finală	T este o interfață	T = B[] este un tablou cu elemente de tipul B
S este o clasă care nu este finală	T trebuie să fie subclasă a lui S	T trebuie să fie o subclasă a lui S	Totdeauna corectă la compilare	S trebuie să fie Object
S este o clasă care este finală	S trebuie să fie subclasă a lui T	T trebuie să fie aceeași clasă ca și S	S trebuie să implementeze interfața T	eroare la compilare
S este o interfață	Totdeauna corectă la compilare	T trebuie să implementeze interfața S	Totdeauna corectă la compilare	eroare la compilare
S = A[] este un tablou cu elemente de tipul A	T trebuie să fie Object	eroare la compilare	eroare la compilare	A sau B sunt același tip primitiv sau A este un tip referință și B poate fi convertit cu un cast la A

Conversii de promovare aritmetică

1. Dacă un operand este double, celălalt este convertit la double.
2. Altfel, dacă un operand este de tip float, celălalt operand este convertit la float.
3. Altfel, dacă un operand este de tip long, celălalt este convertit la long
4. Altfel, amândoi operanzii sunt convertiți la int.

float f;

double i = f + 3;

După efectuarea operației, valoarea obținută va fi convertită implicit la double.

Declararea unei class

package

Class name

Constructor

Fields

methods

Pasul 1: Stabilirea conceptului reprezentat de clasa de obiecte

Pasul 2: Stabilirea numelui clasei de obiecte

Pasul 3: Stabilirea superclasei

Pasul 4: Stabilirea interfețelor pe care le respectă clasa

Pasul 5: Stabilirea modifierilor clasei

- Abstract, final și public.

Pasul 6: Scrierea corpului declarației

Forma generală a unei declarații de clasă

Sintaxa exactă de declarare a unei clase arată în felul următor:

{ abstract | final | public } class *NumeClasă*

[extends *NumeSuperclasă* **]**

[implements *NumeInterfață* **[, NumeInterfață]]**

{ [CâmpClasă] }

Variabilele unei clase

Modificatori

Modificatorii sunt cuvinte rezervate Java care precizează sensul unei declarații. Iată lista acestora:

static

final

transient

volatile

Protecție

În Java există patru grade de protecție pentru o variabilă aparținând unei clase:

- privată
- protejată
- publică
- prietenoasă

Accesarea unei variabile

Accesarea unei variabile declarate în interiorul unei clasei se face folosindu-ne de o expresie de forma:

`ReferințăInstanță.NumeVariabilă`

Referința către o instanță trebuie să fie referință către clasa care conține variabila. Referința poate fi valoarea unei expresii mai complicate, ca de exemplu un element dintr-un tablou de referințe.

În cazul în care avem o variabilă statică, aceasta poate fi accesată și fără să deținem o referință către o instanță a clasei. Sintaxa este, în acest caz:

`NumeClasă.NumeVariabilă`

Vizibilitate

O variabilă poate fi ascunsă de declarația unei alte variabile cu același nume. De exemplu, dacă într-o clasă avem declarată o variabilă cu numele `unu` și într-o subclasă a acesteia avem declarată o variabilă cu același nume, atunci variabila din superclasă este ascunsă de cea din clasă. Totuși, variabila din superclasă există încă și poate fi accesată în mod explicit. Expresia de referire este, în acest caz:

`NumeSuperClasă.NumeVariabilă`

sau

`super.NumeVariabilă`

în cazul în care superclasa este imediată.

La fel, o variabilă a unei clase poate fi ascunsă de o declarație de variabilă dintr-un bloc de instrucțiuni. Orice referință la ea va trebui făcută în mod explicit. Expresia de referire este, în acest caz:

`this.NumeVariabilă`

leșirea unui program Java

Format:

```
System.out.print(<string or variable name one> + <string or variable name two>..);  
SAU  
System.out.println(<string or variable name one> + <string or variable name two>..);
```

Example

```
public class OutputExample1  
{  
    public static void main (String [] args)  
    {  
        int num = 123;  
        System.out.println(„Primul program!");  
        System.out.print(num);  
        System.out.println("num="+num);  
    }  
}
```
